

Self-Reflective RAG

Engineering & Interview Handbook

Retrieval-Augmented Generation with Hybrid Retrieval, LangGraph Self-Reflection, and Benchmark-Driven Configuration

Tech Stack: LangChain · LangGraph · ChromaDB · Ollama (Llama 3.2) · BAAI/bge-small-en-v1.5 · Streamlit

Table of Contents

Table of Contents	2
1. RAG Basics	5
1.1 Data Ingestion.....	5
1.2 Chunking.....	5
1.3 Tokenization	5
1.4 Embeddings	5
1.5 Vector Similarity & ANN Search	6
1.6 Retriever, Prompt Construction & Generation	6
1.7 End-to-End RAG Pipeline	6
1.8 LangChain & LangGraph	7
1.9 Self-Reflective RAG	7
1.10 Evaluation Concepts (Preview)	7
2. Advanced RAG Concepts	8
2.1 Lexical Retrieval: TF-IDF and BM25	8
2.2 Hybrid Retrieval & Reciprocal Rank Fusion	8
2.3 Cross-Encoder Re-ranking	8
2.4 Duplicate Detection with SHA-256	8
2.5 Evolution of RAG Architectures	9
2.6 Production RAG Considerations	9
3. Project Architecture	10
3.1 Tech Stack.....	10
3.2 Architecture Diagram	10
3.3 High-Level Architecture (Narrative)	11
3.4 Key Interview Points	11
4. Pipeline Details	13
4.1 Offline Ingestion Pipeline	13
4.2 Online RAG Pipeline.....	13
4.3 Evaluation Pipeline	14
4.4 Benchmark Pipeline.....	14
5. Benchmarking & Evaluation	15
5.1 Objective.....	15
5.2 Experimental Setup	15
5.3 Benchmark Methodology	15
5.4 Metrics Used.....	15
5.5 Benchmark Results	16

5.6 Statistical Limitations of the Benchmark.....	18
5.7 Full Benchmark Table	19
5.8 Interpretation of Results	19
5.9 Final Configuration Selection.....	19
5.10 Engineering Evaluation Metrics (Production Configuration)	20
5.11 Interpretation	20
5.12 Limitations	20
5.13 Future Evaluation Metrics	21
6. STAR Interview Explanation.....	22
6.1 One-Line Project Summary.....	22
6.2 Situation	22
6.3 Task.....	22
6.4 Action.....	22
6.5 Result.....	23
6.6 One-Sentence Summaries (for quick recall).....	23
7. Interview Questions	24
7.1 Tier 1 — Must Know	24
7.2 Tier 2 — Highly Likely	25
7.3 Tier 3 — Advanced (Topic Checklist)	25
8. Advanced Math & Theory.....	26
8.1 Cosine Similarity	26
8.2 Why Embeddings Capture Meaning.....	26
8.3 Pooling.....	26
8.4 Approximate Nearest Neighbor Search & HNSW	26
8.5 BM25 & IDF, Formally.....	26
8.6 Reciprocal Rank Fusion, Formally.....	27
8.7 Cross-Encoder Mathematics.....	27
8.8 Embedding Dimension Trade-off.....	27
8.9 Why Chunk Overlap Works	27
8.10 Why Large Chunks Hurt Retrieval.....	28
8.11 Hash Collisions.....	28
8.12 Complexity Summary.....	28
8.13 Why RAG Still Hallucinates, and Why Self-RAG Helps.....	28
8.14 Evaluation Metrics, Formally.....	28
8.15 Why Hybrid Retrieval Wins.....	29
8.16 Common Interview Numbers to Remember	29
9. Future Improvements.....	30

9.1 Immediate Improvements 30

9.2 Long-Term Improvements 31

Long-Term Vision..... 32

Appendix — Current Configuration Reference 33

1. RAG Basics

1.1 Data Ingestion

Data ingestion is the process of bringing external data (PDF, Word, Excel, PowerPoint, HTML, CSV, database, website, email, JSON, images) into an AI system so it can answer questions later. Everything eventually becomes text.

- OCR (Optical Character Recognition): converts images of text into machine-readable text — Image → Detect letters → Generate text. Without OCR, an LLM cannot read scanned content.
- PyMuPDF: does not understand language; it reads the native digital structure of a PDF — Page → Objects → Fonts → Coordinates → Text → String. Much faster than OCR for digital PDFs.

1.2 Chunking

A chunk is a small, semantically meaningful portion of a document treated as an independent retrieval unit. There is no universally best chunk size; typical values range from 200–1000 tokens depending on the document and application.

Small Chunk	Large Chunk
Better precision	Better context
May lose information	May mix topics
More vectors	Fewer vectors

Splitting prefers natural boundaries, tried in this order: (1) paragraphs, (2) newlines, (3) spaces, (4) characters as a last resort — this preserves meaning better than blindly cutting every N characters.

A single embedding cannot represent an entire document because large documents span many topics, which reduces retrieval precision. Chunking lets each embedding focus on one concept. Chunk overlap (commonly 20–50 tokens) prevents sentences or ideas that span a chunk boundary from becoming incomplete.

1.3 Tokenization

Tokenization converts text into small units (tokens) and assigns each a unique numerical ID from the model's vocabulary. These token IDs carry no mathematical meaning themselves — they are just labels the model looks up in its embedding matrix. Modern LLMs use subword tokenization (e.g., BPE) because it keeps the vocabulary small while still handling unseen words, unlike pure word-level tokenization.

- Character tokenization — rarely used by modern LLMs.
- Word tokenization — struggles with unknown words and huge vocabularies.
- Subword tokenization (e.g. Byte Pair Encoding) — frequent character combinations merge into single tokens; rare words stay split into pieces. Used in GPT-family models.

1.4 Embeddings

An embedding is a dense numerical vector that captures the semantic meaning of a token or piece of text. Each dimension is learned automatically during training and has no individually interpretable meaning — together the dimensions uniquely characterize the text, the way many small traits together identify a person. Similar meanings produce nearby vectors.

- Token embeddings (inside the LLM): one vector per token, produced via a lookup in a learned embedding matrix — extremely fast, not regenerated at inference time.
- Sentence / document embeddings (used in RAG): a single fixed-length vector representing an entire chunk, produced by pooling contextual token information — this is what is stored in ChromaDB.

Pipeline: Chunk → Tokenizer → Token IDs → Token Embeddings → Transformer Encoder → Pooling → fixed-dimensional vector (e.g. 384-d for BGE-small). ChromaDB stores the chunk text, its embedding, and metadata (author, page number, etc.) — raw token IDs are not stored since they carry no standalone meaning.

1.5 Vector Similarity & ANN Search

Method	Notes
Euclidean Distance	Depends on vector magnitude — not ideal for text embeddings
Dot Product	Larger value = more similar, but long vectors inflate scores
Cosine Similarity	Normalized dot product — the standard choice for embeddings

Comparing a query vector against every stored embedding is expensive at scale, so Approximate Nearest Neighbor (ANN) search is used instead. ChromaDB uses HNSW (Hierarchical Navigable Small World): a multi-layer graph where the top layer has few nodes for fast coarse navigation, and each lower layer refines the search until the nearest vectors are found — without comparing against every stored embedding.

1.6 Retriever, Prompt Construction & Generation

The retriever takes a user question, embeds it, searches ChromaDB by similarity, and returns the top-k most relevant chunks. If k is too small, important context may be missed; if too large, the added noise increases cost without proportional benefit.

Prompt construction combines the retrieved context with the user's question into a structured prompt (System → Context → User Question) before calling the LLM. Restricting the model to the supplied context reduces hallucination. Retrieval quality is often the primary bottleneck in RAG systems; even a stronger LLM cannot reliably compensate for irrelevant retrieved context.

Autoregressive LLMs generate text by predicting the next token conditioned on the previous context, not by verifying whether an answer is true. The model processes the full prompt through attention, then predicts a probability distribution over the next token. Temperature controls randomness (low = deterministic/factual, high = creative); top-p (nucleus sampling) restricts sampling to the smallest set of tokens whose cumulative probability exceeds p. Generation stops at an end-of-sequence token, a stop sequence, or the max-token limit. Because the model has no built-in truth-check, this project adds a self-reflection loop after generation.

1.7 End-to-End RAG Pipeline

PDF → Extract Text → Chunk → Generate Embeddings → Store in ChromaDB → User Question → Query Embedding → Similarity Search → Top-K Chunks → Build Prompt → Call LLM → Answer.

1.8 LangChain & LangGraph

LangChain is an orchestration framework — it does not perform retrieval or generation itself; it wraps and coordinates other components (document loaders, text splitters, embedding clients, vector stores, retrievers) behind a standard interface. This gives three practical advantages: far less boilerplate code, swappable components (e.g. `OpenAIEmbeddings()` → `HuggingFaceEmbeddings()` with everything else unchanged), and a consistent interface across vector databases and embedding models.

A plain LLM pipeline is linear — question in, answer out — with no way to retry, verify, or correct a bad answer. LangGraph is a framework for stateful workflows that can branch, loop, and make decisions during execution using conditional edges. A shared State object (question, retrieved docs, answer, retry count, grade) is read and written by every node, and the graph loops until the grade is "good" or a retry limit is hit.

1.9 Self-Reflective RAG

Self-reflection means the system evaluates its own previous step using a second LLM call, checking three things:

- Document relevance — is this chunk actually relevant to the question? (Garbage in, garbage out.)
- Hallucination / grounding check — is the answer supported by the retrieved documents?
- Answer quality — does the answer actually address the user's question?

If any check fails, the system retries. If retrieval keeps missing relevant context, the query is rewritten and retrieval runs again, rather than retrying with the same failing query indefinitely. A maximum retry / iteration count prevents infinite loops; once exceeded, the system returns a best-effort "I couldn't find enough information" response instead.

Normal RAG	Self-RAG
Retrieve once	Can retrieve multiple times
No verification	Verifies retrieval and answer
Linear pipeline	Graph with loops
Lower cost, faster	Higher cost, slower
More hallucinations	Fewer hallucinations

1.10 Evaluation Concepts (Preview)

Standard ML evaluation does not transfer well to RAG, since different wordings of an answer can both be correct. Four concepts recur throughout RAG literature and RAGAS in particular — Faithfulness, Answer Relevance, Context Precision, and Context Recall. These are covered in depth in the Benchmarking & Evaluation chapter.

2. Advanced RAG Concepts

2.1 Lexical Retrieval: TF-IDF and BM25

Semantic search is strong at concepts, meaning, and paraphrase, but weak on exact identifiers — a user searching for "YOLOv10n" wants that exact term, not something merely similar. Lexical search based on the literal words is better suited to error codes, names, versions, APIs, and technical documentation.

TF-IDF scores a document by Term Frequency (how often a word appears) multiplied by Inverse Document Frequency (how rare that word is across the whole corpus). Its weaknesses: term-frequency contribution grows roughly linearly with no diminishing return, and it does not normalize for document length — a 100-page document racks up more raw occurrences than a 2-page one regardless of relevance. BM25 fixes both: it saturates term frequency so repeated occurrences add progressively less, and it normalizes for document length so long documents are not unfairly favored.

2.2 Hybrid Retrieval & Reciprocal Rank Fusion

Hybrid retrieval runs semantic search (over embeddings) and BM25 (over an inverted index) in parallel against the same corpus and combines the two ranked lists:

Question → [Semantic Search (Vector DB)] + [BM25 Search (Keyword)] → Combine Results → Better Documents

Because semantic similarity scores (e.g. 0.82) and BM25 scores (e.g. 17.8) live on incompatible scales, a simple average is meaningless. Reciprocal Rank Fusion (RRF) instead trusts each document's rank position in each list, not its raw score:

$$RRF(d) = \sum 1 / (k + rank_i(d))$$

A typical constant is $k \approx 60$ — large enough that a single retriever can't dominate, small enough that top-ranked documents still matter. RRF ignores confidence scores entirely, trading a little information for scale-independence.

2.3 Cross-Encoder Re-ranking

A bi-encoder (standard embedding model) encodes the query and each document independently, then compares vectors via cosine similarity — the query never "sees" the document, which makes it very fast but less precise. A cross-encoder concatenates query and document and passes them through a transformer together, letting query tokens attend directly to document tokens (query–document cross-attention) and outputting a single relevance score. Cross-encoders generally achieve higher reranking accuracy than bi-encoders because they jointly encode the query and document, but this requires one full transformer forward pass per pair, so it is only affordable over a small candidate set: Hybrid Retrieval → Top-20 candidates → Cross-Encoder → Top-5 final documents. No storage is required for a cross-encoder since nothing is pre-computed.

2.4 Duplicate Detection with SHA-256

SHA-256 converts an arbitrary-sized file into a fixed-length 256-bit digest. It is deterministic (same input → same output) and exhibits the avalanche effect (a one-character change produces a completely different hash). Computing a hash is far cheaper than re-chunking, re-embedding, and re-inserting a document, so it is used to

skip re-ingesting a PDF that has already been indexed. A hash collision is theoretically possible but, at 2^{256} possible outputs, practically treated as impossible.

2.5 Evolution of RAG Architectures

Architecture	Key Idea
Naive RAG	Retrieve once, generate once — fast, cheap, but no retry or verification
Advanced RAG	Adds better chunking, metadata, hybrid search, re-ranking — still no feedback loop
Self-RAG	Retrieve → Generate → Evaluate → Retry — the model checks itself
Agentic RAG	An LLM agent reasons about the task and chooses among multiple tools (retrievers, APIs, calculators)
CRAI (Corrective RAG)	Evaluates retrieval quality and falls back to alternative retrieval / web search if poor
GraphRAG	Retrieves relationships from a knowledge graph instead of flat chunks

2.6 Production RAG Considerations

- Document changes: only the modified document's chunks are re-embedded and the vector store updated (re-indexing), not the whole corpus.
- Duplicate uploads: detected via document hash / file ID, and metadata filtering speeds up retrieval.
- Cost control: query embeddings are cheap and generated per-search; document embeddings are generated once at ingestion. Caching (prompt, embedding, and LLM-response caches) avoids paying for repeated questions.
- Scale: beyond a single ChromaDB instance, distributed vector databases (Pinecone, Weaviate, Milvus, Qdrant) support horizontal scaling across machines.
- Security: requests flow through Authentication → Permission Check → Retriever.
- Freshness: requires continuous re-indexing as source documents change.
- Latency: optimized via ANN indexing, better hardware, caching, smaller models, and parallel retrieval.
- Observability & hallucination monitoring: logging at every stage plus automatic evaluation (Self-RAG style checks, or RAGAS) since failures are otherwise invisible.

Full production flow: Documents → Ingestion → Chunking → Embeddings → Vector DB → Monitoring → Caching → Metadata → Retriever → Prompt → LLM → Evaluation → Logging.

3. Project Architecture

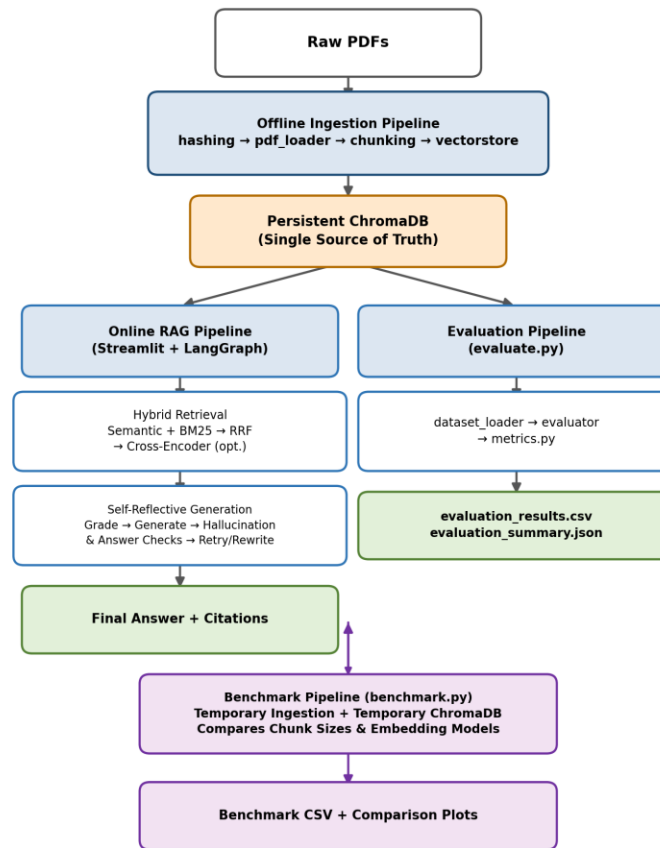
3.1 Tech Stack

Layer	Choice
LLM	Ollama — Llama 3.2 (local inference)
Orchestration	LangChain + LangGraph
Frontend	Streamlit
Vector Database	ChromaDB
Retrieval	Semantic + BM25 + RRF, optional Cross-Encoder re-ranking
Embeddings	HuggingFace Embeddings — BAAI/bge-small-en-v1.5 (384-d)

3.2 Architecture Diagram

The system separates offline document preparation from online question answering, sharing a single persistent vector store between them. The Benchmark Pipeline is deliberately isolated — it builds its own temporary vector stores and never touches the production index.

Self-Reflective RAG — System Architecture



Isolated from production index — never writes to the persistent ChromaDB

End-to-end system architecture: offline ingestion feeds a persistent ChromaDB, which serves both the online RAG pipeline and the evaluation pipeline; the benchmark pipeline runs in isolation.

3.3 High-Level Architecture (Narrative)

RAW PDFs → OFFLINE INGESTION PIPELINE (hashing → PDF loading → chunking → vectorstore) → Persistent ChromaDB ↓ (branches into) ONLINE RAG PIPELINE (Streamlit → LangGraph → Hybrid Retrieval → Self-Reflective Generation → Final Answer + Citations) EVALUATION PIPELINE (evaluate.py → dataset_loader → evaluator → metrics → CSV/JSON reports) A separate BENCHMARK PIPELINE builds temporary ingestion + temporary Chroma stores to compare chunk sizes and embedding models without touching the production index.

3.4 Key Interview Points

- Offline ingestion and online inference are completely separated.
- ChromaDB is the single source of truth.
- Hybrid Retrieval uses RRF because semantic and BM25 scores are not directly comparable.
- Cross-Encoder only reranks the top candidates, avoiding scoring the entire corpus.
- LangGraph manages retries, query rewriting, hallucination checking, and answer usefulness.

- Evaluation uses the same production retriever and workflow, ensuring realistic measurements.
- Benchmarking rebuilds temporary vector stores per configuration, preventing contamination of the production index.
- Evaluation is fully deterministic (latencies, counts, page recall) — it does not rely on an LLM judge.

4. Pipeline Details

4.1 Offline Ingestion Pipeline

Purpose: convert raw PDF documents into searchable vector embeddings stored inside ChromaDB.

Run: `python ingest.py`

Module	Responsibility
hashing.py	Compute SHA-256 hash, compare with manifest.json, skip duplicate PDFs
pdf_loader.py	Open PDF with PyMuPDF, extract page text, return Page objects
chunking.py	Split pages into chunks, detect section headings, attach metadata (document_id, chunk_id, page, filename, source, section, created_at)
vectorstore.py	Sanitize metadata, generate embeddings via LangChain, store vectors in ChromaDB, update manifest.json

Output flow: Raw PDFs → Chunked Documents → Embeddings → Persistent ChromaDB.

4.2 Online RAG Pipeline

Purpose: answer user questions using the indexed knowledge base.

Run: `streamlit run app.py`

User Question → Streamlit UI → Compiled LangGraph Workflow → `retrieve()` → Hybrid Retrieval (Semantic + BM25 → RRF → optional Cross-Encoder) → `grade_documents()` → `generate()` → Hallucination Grader → Answer Grader → (Good Answer → END) | (Rewrite Query / Retry Generation → loop) → `accept_best_effort()` on exhausted retries → Final Answer + Citations.

LangGraph Nodes

Node	Responsibility
<code>retrieve()</code>	Calls the retriever, returns documents
<code>grade_documents()</code>	Grades each chunk (question + chunk → relevant?), keeps only filtered_docs
<code>decide_to_generate()</code>	Routes to <code>transform_query</code> if filtered_docs is empty, else generate
<code>transform_query()</code>	Calls the question_rewriter, updates the question in state
<code>generate()</code>	Builds context + prompt, calls Ollama, produces answer with citations (filename, page, section)
<code>grade_generation_v_documents_and_question()</code>	Checks hallucination and answer usefulness; routes to END, retry, rewrite, or best-effort

Node	Responsibility
accept_best_effort()	Sets low_confidence = True and returns the current answer once retries are exhausted

GraphState Evolution

question → (+documents after retrieve) → (+generation after generate) → (+generation_attempts during hallucination grading) → (+query_rewrite_attempts during rewriting) → final state: question, documents, generation, sources, low_confidence.

4.3 Evaluation Pipeline

Purpose: measure the performance of the production RAG pipeline using deterministic engineering metrics.

Run: `python evaluate.py`

Flow: parse CLI args → load Chroma vector store → `dataset_loader.py` reads and validates `questions.json` → build production retriever + LangGraph → `evaluator.py` times retrieval and the full workflow, collects the final GraphState, and measures retries/citations → `metrics.py` computes page recall and averages → writes `evaluation_results.csv` and `evaluation_summary.json`.

Metrics Computed

Category	Metrics
Retrieval	Retrieval latency, retrieved chunks, reranked chunks, retrieved pages, expected pages, page recall
Generation	Generation latency, workflow latency, answer length, retry count, query rewrite count, low-confidence flag
Citations	Citation count, citation pages, duplicate citations removed

Page recall per question is computed as: Expected Pages → Retrieved Pages → Intersection → Recall.

4.4 Benchmark Pipeline

Purpose: compare different chunk sizes and embedding models to select the best RAG configuration.

Run: `python benchmark.py`

For each chunk-size configuration, `run_config()` builds a temporary directory, runs a temporary ingestion into a temporary Chroma store, builds a retriever, benchmarks retrieval and generation, and returns a `BenchmarkResult` — measuring number of chunks, ingestion time, retrieval latency, generation latency, workflow latency (retrieval + generation), and page recall. Results are written to CSV and comparison plots are generated. This design keeps benchmarking fully isolated from the production index (see Chapter 5 for full results).

5. Benchmarking & Evaluation

5.1 Objective

The benchmark was conducted to identify an optimal chunk size that balances retrieval quality, latency, and storage efficiency. Nine chunk sizes were evaluated against the same document corpus and the same 51-question evaluation dataset, holding every other pipeline component fixed.

5.2 Experimental Setup

Parameter	Value
Embedding Model	BAAI/bge-small-en-v1.5
LLM	Llama 3.2 (via Ollama)
Retriever	Hybrid Retrieval (BM25 + Semantic + RRF)
Re-ranking	Cross-Encoder (ms-marco-MiniLM-L-6-v2)
Evaluation Dataset	51 questions
Chunk Overlap	50 tokens
Chunk Sizes Tested	150, 200, 250, 300, 350, 400, 500, 600, 800

5.3 Benchmark Methodology

To keep the comparison fair, each chunk size was evaluated as a fully isolated run against a temporary vector store, then torn down before the next configuration began:

- Delete any existing temporary vector database.
- Re-ingest the source documents at the target chunk size.
- Generate embeddings for the resulting chunks.
- Build the retriever for this configuration.
- Run the complete 51-question evaluation dataset.
- Record all metrics (chunks, timings, page recall).
- Delete the temporary database before moving to the next chunk size.

This guarantees no cross-contamination between configurations and that every chunk size is measured under identical conditions — the same documents, same questions, same retrieval and generation logic.

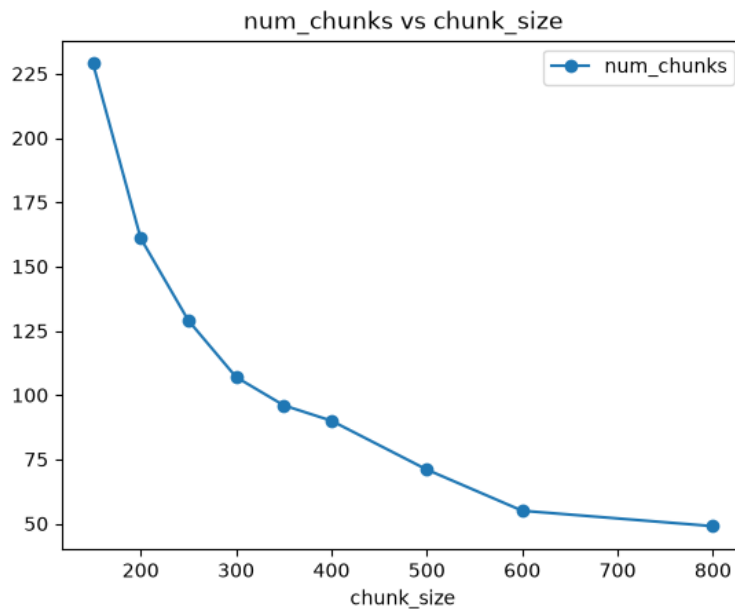
5.4 Metrics Used

Metric	What it Measures
Number of Chunks	How many chunks the corpus is split into at a given chunk size
Ingestion Time	Time to extract, chunk, embed, and store the corpus
Retrieval Latency	Time to run hybrid retrieval + re-ranking for one query

Metric	What it Measures
Generation Latency	Time for the LLM to produce an answer from retrieved context
Workflow Latency	Retrieval + generation combined, end-to-end per question
Average Page Recall	Fraction of expected source pages actually retrieved

5.5 Benchmark Results

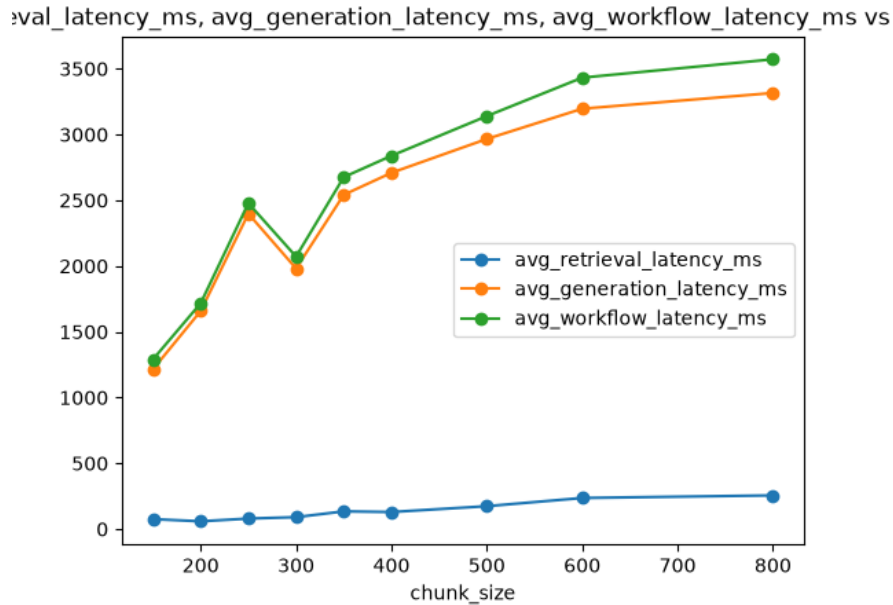
Figure 1 — Chunk Size vs. Number of Chunks



Number of chunks falls sharply as chunk size increases — from 229 chunks at size 150 to 49 chunks at size 800.

Observation: increasing chunk size reduces the number of generated chunks in a clear, roughly inverse relationship — a natural consequence of packing more text into each chunk.

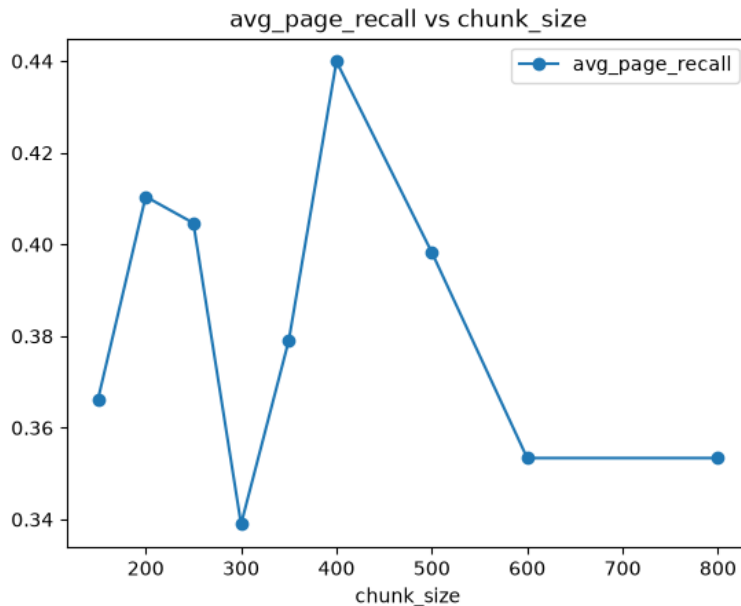
Figure 2 — Chunk Size vs. Retrieval / Generation / Workflow Latency



Retrieval latency stays low and flat; generation and workflow latency climb steadily as chunk size grows.

Observation: retrieval latency increases only slightly with chunk size (75 ms → 256 ms across the full range), since it mainly depends on index size and re-ranking cost. Generation latency grows far more (1.2 s → 3.3 s) because larger chunks mean more context tokens fed to the LLM, which dominates total workflow latency.

Figure 3 — Chunk Size vs. Average Page Recall



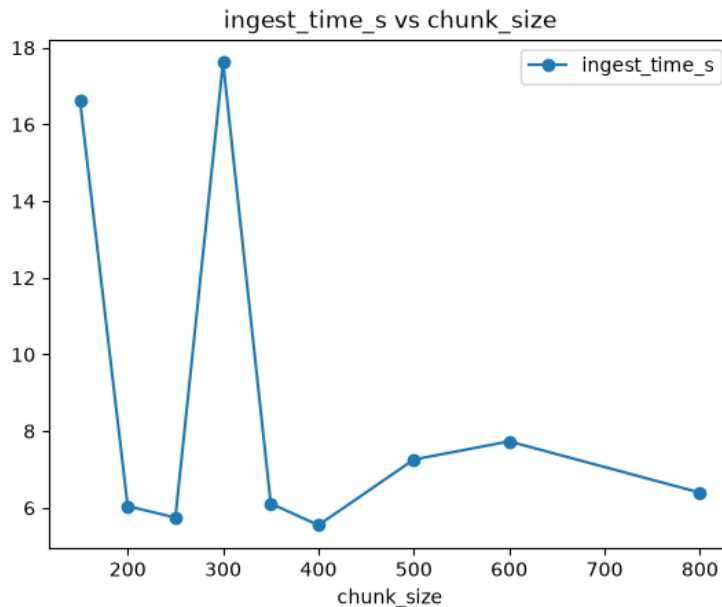
Page recall peaks at chunk size 400 (0.44) and declines for both very small and very large chunks.

The benchmark demonstrates that retrieval quality does not improve monotonically with chunk size. Very small chunks lose surrounding context, fragmenting relevant information across multiple retrieval units; very large

chunks dilute semantic meaning by mixing multiple topics into a single embedding. Chunk size 400 achieved the best trade-off between contextual completeness and retrieval precision, peaking at 0.44 average page recall.

The noticeable drop at chunk size 300 (0.339, well below both its 250 and 350 neighbors) is a clear outlier in this curve. It suggests that this specific configuration fragmented relevant information more than neighboring chunk sizes — possibly because a 300-token boundary happened to split key passages awkwardly for this document set. Further experiments (ideally repeated runs, see Section 5.6) would be required to determine whether this reflects a genuine document-structure effect or benchmark variance.

Figure 4 — Chunk Size vs. Ingestion Time



Ingestion time is mostly stable (5.5–7.7 s) with two isolated spikes at chunk sizes 150 and 300.

Observation: ingestion time remains relatively stable except for isolated spikes at chunk sizes 150 (16.6 s) and 300 (17.6 s), most likely caused by embedding-generation variability or system-level noise (e.g. background load) rather than a real effect of chunk size — this should be confirmed by rerunning those two configurations before drawing conclusions from them. Notably, chunk size 300 shows an anomaly in both ingestion time and page recall, which strengthens the case that this configuration was affected by transient noise rather than being a stable, reproducible data point.

5.6 Statistical Limitations of the Benchmark

The benchmark was executed once per configuration rather than averaged across repeated runs. This is an important caveat when reading the results above:

- A single run cannot distinguish a genuine chunk-size effect from transient system noise (background CPU load, disk I/O, embedding-model warm-up, etc.).
- Multiple benchmark runs per chunk size, with mean and variance reported, would produce more statistically reliable latency and recall measurements by averaging out this noise.

- This matters concretely here: the ingestion-time spikes at chunk sizes 150 and 300, and the page-recall dip at 300, are exactly the kind of single-run artifacts that repeated trials would help confirm or rule out.

In a production setting, this benchmark would be re-run 3–5 times per configuration, reporting the mean and standard deviation for each metric, before finalizing a chunk-size decision.

5.7 Full Benchmark Table

Chunk Size	Chunks	Ingest (s)	Retrieval (ms)	Generation (ms)	Workflow (ms)	Page Recall
150	229	16.62	75.3	1214.9	1290.3	0.366
200	161	6.04	58.9	1658.1	1717.1	0.410
250	129	5.75	79.7	2398.0	2477.8	0.405
300	107	17.62	90.3	1981.6	2071.9	0.339
350	96	6.11	134.5	2541.7	2676.2	0.379
400	90	5.55	129.5	2709.0	2838.5	0.440
500	71	7.26	173.9	2966.3	3140.2	0.398
600	55	7.73	236.2	3196.2	3432.4	0.353
800	49	6.40	256.0	3316.0	3572.0	0.353

5.8 Interpretation of Results

- Smaller chunks (150–250) generate more chunks and keep generation latency lower, but retrieval precision suffers from fragmented, incomplete context — recall stays around 0.37–0.41.
- Chunk size 400 gives the highest page recall (0.44) in the sweep, with generation latency (≈ 2.7 s) still well below the largest chunk sizes.
- Beyond 400, larger chunks reduce the number of retrievable units and dilute each chunk's topical focus, so recall falls to ~ 0.35 even though each chunk carries more raw text.
- Retrieval latency scales gently with chunk size throughout (since it is dominated by index lookup, not chunk length), while generation latency scales with chunk size much more steeply, since larger chunks mean more tokens per LLM call.

5.9 Final Configuration Selection

Selected chunk size: 400 tokens (overlap 50)

- Highest page recall (0.44) among all nine configurations tested.
- Reasonable latency — generation and workflow latency remain mid-pack, well below the 600–800 range.
- Strong balance between retrieval accuracy, chunk count (storage/index size), and end-to-end latency.
- Suitable for production deployment — this is the configuration used for the production evaluation run in Section 5.10.

This is a materially stronger justification than simply asserting "400 was chosen" — it is backed by a controlled sweep across nine configurations on a fixed evaluation set.

5.10 Engineering Evaluation Metrics (Production Configuration)

With chunk size 400 selected, the full production pipeline (hybrid retrieval + cross-encoder re-ranking + LangGraph self-reflection) was evaluated end-to-end on the same 51-question set:

Metric	Value
Questions evaluated	60
Average retrieval latency	133.94 ms
Average generation latency	7187.20 ms
Average workflow latency	7321.14 ms
Average retrieved / reranked chunks	4.0 / 4.0
Average page recall	0.5582
Retry rate	21.67%
Query rewrite rate	16.67%
Low-confidence rate	11.67%
Average citations per answer	3.33

Note the production retrieval latency (125 ms) and generation latency (~9.7 s) are considerably higher than the equivalent chunk-400 benchmark row above — this reflects the added cost of cross-encoder re-ranking and the LangGraph self-reflection loop (grading, hallucination checking, and possible retries), which the raw benchmark sweep does not include.

5.11 Interpretation

- An average page recall of 0.56 means the retriever surfaces roughly 56% of the expected supporting pages on average — leaving clear room for improvement (see Chapter 9).
- Retrieval latency (134 ms) is relatively low compared to generation latency (7.2 s) — generation dominates total workflow time, so latency optimization efforts should target the LLM call and the self-reflection loop rather than retrieval.
- A retry rate of 22% shows the self-reflective workflow actively corrects low-quality retrievals — nearly one fifth of all questions triggered at least one retry, demonstrating that the system actively attempts to improve answer quality rather than returning the first generated response unchecked.
- A 12% low-confidence rate reflects conservative behavior: the system flags uncertainty instead of confidently hallucinating an answer when it cannot ground one well.
- An average of 3.33 citations per answer indicates the system usually grounds its responses using multiple supporting sources rather than a single chunk, which is a good signal for answer reliability.

5.12 Limitations

- Evaluation was run against a single PDF corpus.
- The benchmark dataset (51 questions) is relatively small.
- Each chunk-size configuration was benchmarked with a single run rather than repeated trials (see Section 5.6).
- Only Page Recall was measured — no Precision@K, MRR, or nDCG.
- No RAGAS-style comparison (Faithfulness, Context Precision/Recall, Answer Relevancy) was run in this evaluation pass.

5.13 Future Evaluation Metrics

- Precision@K and Recall@K
- Mean Reciprocal Rank (MRR)
- Normalized Discounted Cumulative Gain (nDCG)
- RAGAS: Faithfulness, Context Precision, Context Recall, Answer Relevancy, Answer Correctness

6. STAR Interview Explanation

6.1 One-Line Project Summary

A Self-Reflective Retrieval-Augmented Generation system for document question answering: a user uploads a PDF and asks natural-language questions; instead of relying on the LLM's internal knowledge, the system retrieves relevant context from the document and generates answers grounded in that context, with a self-reflection loop that checks and, if needed, retries retrieval and generation before returning an answer.

6.2 Situation

While experimenting with LLMs for document question answering, I noticed the models often leaned on pre-trained knowledge instead of the uploaded PDF, producing answers that were sometimes incorrect or ungrounded. Researching this led me to Retrieval-Augmented Generation. Rather than use an off-the-shelf implementation, I built a self-reflective RAG system from scratch to understand the full pipeline and improve answer reliability.

6.3 Task

Design a RAG system that could retrieve relevant information from a document, detect poor retrieval, automatically improve the query, verify that a generated answer is actually supported by the retrieved context, retry when necessary, and return only a well-grounded answer — with a workflow that stayed modular, visualizable, and easy to extend.

6.4 Action

- Document processing: PDF → text (PyMuPDF) → overlapping chunks, preserving context across chunk boundaries.
- Embeddings: each chunk converted to a dense embedding and stored inside ChromaDB.
- Hybrid retrieval: semantic search (embeddings) combined with BM25 (keywords) via Reciprocal Rank Fusion, since some questions depend on meaning and others on exact terms.
- Document grading: an LLM grades each retrieved chunk as relevant or not before it reaches the generator; irrelevant chunks are discarded.
- Query rewriting: if too few relevant chunks survive grading, another LLM rewrites the query and retrieval runs again — a self-correcting loop.
- Answer generation: the local LLM (via Ollama) generates an answer using only the retrieved, graded context.
- Self-reflection: a hallucination check verifies every important claim is grounded in the retrieved documents, and an answer-quality check verifies the answer actually addresses the question; either failure re-triggers retrieval or rewriting.
- Orchestration: LangGraph represents each stage (retrieve, grade, rewrite, generate, evaluate) as a node, with evaluation results deciding which edge to follow next — replacing brittle nested if/else logic.

- Frontend: a Streamlit interface for uploading documents, asking questions, and observing retrieval and generation interactively.

6.5 Result

- Reduced hallucinations through answer verification.
- Improved retrieval quality through automatic query rewriting.
- Filtered irrelevant retrieved documents before generation.
- More reliable, document-grounded answers with citations.
- An interactive interface and a modular architecture that can incorporate stronger retrieval or larger LLMs later.

6.6 One-Sentence Summaries (for quick recall)

- Chunking: each chunk is split with overlap so information spanning a boundary isn't lost.
- Embeddings: each chunk is converted into a dense embedding and stored inside ChromaDB.
- LangGraph: manages the retry/rewrite/verify loop as a graph instead of nested conditionals.

7. Interview Questions

7.1 Tier 1 — Must Know

Document Processing

- Why can't an LLM directly process the PDF? — It needs embeddings for fast, repeatable retrieval and storage, instead of reprocessing the raw document for every query.

Embeddings

- How are embeddings generated? — Via a Sentence Transformer model, over the chunk's tokens.
- Which embedding model did you use? — BAAI/bge-small-en-v1.5.
- What is the dimensionality of your embeddings? — 384 dimensions.
- What does each dimension represent? — Each of the 384 dimensions is a learned latent feature with no single human-readable meaning; semantic meaning is distributed across the whole vector, so similar text produces nearby vectors in that 384-dimensional space.

ChromaDB

- Why ChromaDB? — Open-source, integrates easily with LangChain, well suited to local RAG apps; stores embeddings with metadata and supports efficient similarity search without external services.
- Why not SQLite? — It has no native support for efficient vector similarity search.

Hybrid Retrieval

- How many chunks do you retrieve? — 4 final chunks after re-ranking, drawn from a candidate pool of 10 from hybrid retrieval.

Document Grading

- What prompt did you use? — The grader outputs a simple binary decision (yes/no), which the LangGraph workflow uses to filter irrelevant chunks before generation.

Answer Generation

- Which LLM did you use? — Llama 3.2, run locally via Ollama.
- Why Ollama? — Simple local serving of open-source LLMs, an API compatible with LangChain, easy model switching, no API cost, and no internet dependency — ideal for local development.

Project Results

- What are the limitations? — The biggest is latency: document grading, query rewriting, hallucination checking, and answer evaluation each require additional LLM calls before a final answer is produced.
- What would you improve? — Semantic chunking instead of fixed-size chunks; FastAPI + Docker deployment; caching for embeddings and LLM responses to cut latency.
- What was the hardest part? — Designing the self-reflective workflow: coordinating retrieval, grading, query rewriting, generation, and evaluation with careful, loop-safe control flow.

7.2 Tier 2 — Highly Likely

- How does PyMuPDF extract text? — A PDF stores text objects, font info, layout info, and drawing instructions rather than flowed text like a Word document. PyMuPDF parses this structure, locates text objects per page, reconstructs reading order, and returns the extracted text for chunking and embedding.
- Explain RecursiveCharacterTextSplitter — A LangChain splitter that recursively tries separators in order (paragraphs, newlines, sentences, spaces, characters) rather than cutting at a fixed character count, producing cleaner, more semantically coherent chunks.
- What indexing algorithm does ChromaDB use? — HNSW (Hierarchical Navigable Small World): a multi-layer graph enabling fast approximate nearest-neighbor search without scanning every embedding.
- What are the disadvantages of Ollama? — Local models are generally smaller/less capable than top cloud models, require local CPU/RAM/GPU resources, are usually slower than optimized cloud APIs, and may underperform on complex reasoning.
- How many retries do you allow? — 2 hallucination retries and 2 query-rewrite retries.

7.3 Tier 3 — Advanced (Topic Checklist)

These are commonly asked by ML-focused or senior interviewers. Full explanations for the starred (★) topics are in Chapter 8 — Advanced Math & Theory; the remainder are natural follow-on questions from those same concepts.

- Transformer embeddings, self-attention, positional encoding, and tokenization (BPE / WordPiece) ★
- How Sentence Transformers work internally; pooling strategy (CLS vs. mean pooling) ★
- BM25 derivation, IDF, and length normalization mathematically ★
- Cosine similarity mathematics and why vectors are normalized ★
- Dense vs. sparse retrieval; encoder-only vs. decoder-only models; why encoders are used for embeddings; contrastive learning and how embedding models are trained
- ANN search, HNSW, IVF, and Product Quantization ★
- Cross-Encoder architecture, why it's slower than a bi-encoder, and reranking mathematics ★
- Retrieval metrics: Precision@K, Recall@K, MRR, nDCG
- RAG evaluation — retrieval evaluation vs. generation evaluation
- RAG vs. fine-tuning; RAG vs. long-context models
- Scaling to millions of documents: multi-document / multi-PDF indexing, distributed vector databases
- Comparisons: FAISS vs. ChromaDB, Pinecone vs. ChromaDB, Weaviate vs. ChromaDB, Milvus vs. ChromaDB, Elasticsearch vs. BM25
- Production concerns: hybrid retrieval at scale, caching embeddings, incremental indexing, streaming ingestion, async/parallel retrieval, and deployment strategy

8. Advanced Math & Theory

8.1 Cosine Similarity

Euclidean distance depends on vector magnitude, not just direction. Example: vectors (100,100) and (101,101) are almost the same direction and have Euclidean distance ≈ 1.41 ; vectors (1,1) and (2,2) have the same distance (≈ 1.41) despite a very different semantic relationship. For embeddings, what matters is direction, not length.

Dot product: $A \cdot B = \sum A_i B_i$ — larger values suggest similarity, but longer vectors naturally produce larger dot products, so magnitude distorts the score.

Cosine similarity normalizes this out: $\text{Cosine Similarity} = (A \cdot B) / (||A|| ||B||)$, where $||A|| = \sqrt{\sum A_i^2}$. Range: +1 (identical direction), 0 (unrelated), -1 (opposite). Since embeddings are normalized, only orientation matters — and most embedding models are trained assuming cosine similarity as the comparison metric.

Interview answer: cosine similarity removes the effect of vector magnitude and measures only semantic direction; since embedding magnitude carries little semantic information, cosine similarity gives more reliable retrieval.

8.2 Why Embeddings Capture Meaning

Embedding models are trained so that similar sentences produce nearby vectors — e.g. "The dog is running" and "A puppy is running" land close together, while "Today's stock market crashed" lands far away. This is learned entirely from data during pretraining; there are no hand-written rules.

8.3 Pooling

A transformer outputs one embedding per token, but retrieval needs one vector per chunk. Pooling reduces many token embeddings into one:

- Mean pooling: average of all token vectors — the most common choice for sentence embedding models.
- CLS pooling: uses only the special [CLS] token's embedding.
- Max pooling: keeps the maximum value per dimension across all tokens.

8.4 Approximate Nearest Neighbor Search & HNSW

Comparing a query against every one of N stored embeddings is $O(N)$ — too slow at scale (e.g. 10 million embeddings). ANN search avoids checking every vector, bringing typical search time down to roughly $O(\log N)$, with the exact complexity depending on the specific ANN algorithm.

HNSW builds multiple graph layers: a sparse top layer for fast coarse movement, down through middle layers, to a dense bottom layer holding the nearest neighbors. Instead of checking every embedding, the search "walks" through the graph. This trades a small amount of recall for a large gain in speed (or vice versa, depending on tuning) rather than performing an exhaustive scan.

8.5 BM25 & IDF, Formally

$$BM25 = IDF \times [TF \cdot (k1+1)] / [TF + k1 \cdot (1 - b + b \cdot (dl / avgdl))]$$

Symbol	Meaning
TF	Term frequency in the document
IDF	Inverse document frequency
dl	Document length
avgdl	Average document length across the corpus
k1	Controls TF saturation (typically 1.2–2.0)
b	Controls document-length normalization (typically ≈0.75)

Unlike TF-IDF, whose score keeps climbing as a term repeats, BM25's TF term saturates — repeated occurrences eventually contribute very little extra score — and it explicitly penalizes documents that are unusually long relative to the corpus average.

$IDF(\text{term}) = \log(N / df)$, where N is the total number of documents and df is the number of documents containing the term. Common words get a low IDF; rare words get a high IDF.

8.6 Reciprocal Rank Fusion, Formally

$RRF(d) = \sum_i 1 / (k + \text{rank}_i(d))$, typically with $k = 60$ — large enough that no single retriever dominates, small enough that top-ranked documents still matter. RRF is used instead of averaging raw scores because semantic similarity (e.g. 0.82) and BM25 scores (e.g. 17.8) are on incompatible scales; ranks are directly comparable, scores are not.

8.7 Cross-Encoder Mathematics

Bi-encoder: Query → Embedding, Document → Embedding (independently), compared via cosine similarity — cheap to reuse since document embeddings are stored once. Cross-encoder: Query + Document → Transformer → single relevance score. Cross-Encoders generally achieve higher reranking accuracy than bi-encoders because they jointly encode the query and document, letting query tokens attend directly to document tokens — but at the cost of one full forward pass per query–document pair.

Why it doesn't scale to the full corpus: over 100,000 chunks, a bi-encoder needs one query embedding and 100,000 cheap cosine comparisons, while a cross-encoder would require 100,000 transformer forward passes — infeasible in production. The practical compromise: Hybrid Retrieval → Top-20 candidates → Cross-Encoder → Top-5 final documents.

8.8 Embedding Dimension Trade-off

Typical dimensions: 384, 768, 1024, 1536. Higher dimensions capture richer representations but cost more storage and slower retrieval; smaller dimensions (e.g. BGE-small's 384) are faster with a modest quality trade-off — a reasonable choice for a local, latency-sensitive project.

8.9 Why Chunk Overlap Works

Without overlap, a sentence like "The algorithm first computes the gradient before updating weights" can be split mid-thought across two chunks, losing meaning at the boundary. With overlap, the boundary text appears in both chunks, so the idea survives intact in at least one retrievable unit.

8.10 Why Large Chunks Hurt Retrieval

A large chunk spanning multiple unrelated topics (e.g. deep learning, networking, operating systems, databases in one chunk) produces one diluted embedding that represents all of them poorly. Smaller, single-topic chunks produce cleaner, more discriminative embeddings — but chunks that are too small lose surrounding context, which is exactly the trade-off the chunk-size benchmark in Chapter 5 quantifies.

8.11 Hash Collisions

SHA-256 has 2^{256} possible outputs; the probability of two different files colliding is astronomically small, which is why production systems treat a SHA-256 match as a reliable indicator that a file has already been processed.

8.12 Complexity Summary

Operation	Complexity
Exact search	$O(N)$
ANN search	$\sim O(\log N)$
BM25 retrieval	Sub-linear in practice (inverted index)
Cross-encoder re-ranking	$O(K \times \text{transformer forward pass})$
Embedding generation	$O(\text{tokens})$
Chunking	$O(\text{document size})$

8.13 Why RAG Still Hallucinates, and Why Self-RAG Helps

An LLM predicts the next token, not truth. Even with retrieved context, wrong chunks, incomplete chunks, or an ambiguous prompt can still lead to hallucination — hence the self-reflection loop: Grounding Check → Retry. Traditional RAG stops after Retrieve → Generate; Self-RAG adds Evaluate → Retry → Rewrite → Generate Again, checking whether retrieval was relevant, whether the answer is grounded in the retrieved documents, and whether it actually answers the question, rather than trusting the first pass.

8.14 Evaluation Metrics, Formally

Metric	Definition
Faithfulness	Is the answer supported by the retrieved context? (Detects hallucination.)
Context Precision	Relevant Retrieved Chunks / Retrieved Chunks — measures retrieval quality.
Context Recall	Relevant Retrieved Chunks / All Relevant Chunks — measures whether important information was missed.
Answer Relevance	Does the generated answer actually address the user's question?

8.15 Why Hybrid Retrieval Wins

	Strengths	Weaknesses
Semantic Search	Synonyms, paraphrases, concepts	API names, version numbers, error codes
BM25	Exact keywords, identifiers, technical terms	Cannot understand meaning

Hybrid retrieval combines both to improve overall recall while preserving precision — each method covers the other's blind spot.

8.16 Common Interview Numbers to Remember

Item	Typical Value
Chunk size	200–1000 tokens
Chunk overlap	20–50 tokens
Top-k retrieval	3–10
Cross-encoder candidates	20–50
Final documents	3–5
RRF k	60
BGE-small dimension	384
OpenAI text-embedding-3-small dimension	1536

9. Future Improvements

The current pipeline performs well on factual question answering (0.44 peak page recall at chunk size 400), but several improvements would push retrieval accuracy, answer quality, latency, and robustness further. These are grouped below into changes that could be made quickly with the existing architecture, and larger, longer-horizon changes.

9.1 Immediate Improvements

Achievable within the current architecture, without a fundamental redesign of the pipeline.

1. Adaptive Chunking (Semantic Chunking)

Improvements: Semantic chunking based on headings/paragraphs; dynamic chunk size by document structure; keep tables/lists intact as single chunks; merge very small chunks.

Expected benefit: Better retrieval precision, improved context quality, higher answer completeness.

2. Better Query Rewriting (incl. HyDE)

Improvements: Detect ambiguous queries; expand abbreviations; generate multiple rewrites and merge results (multi-query retrieval); use HyDE (Hypothetical Document Embeddings) for difficult queries.

Expected benefit: Higher recall, better handling of vague questions, fewer retrieval failures.

3. Metadata-Aware Retrieval (Filtering)

Improvements: Use section, chapter, heading, page, and document type as pre-filters before semantic search.

Expected benefit: Better precision, faster search, fewer irrelevant chunks.

4. Caching

Improvements: Prompt cache, embedding cache, and LLM-response cache for repeated or similar questions.

Expected benefit: Lower latency, lower LLM cost, less redundant computation.

5. OCR Support

Improvements: Tesseract or PaddleOCR; table extraction; scanned-document support.

Expected benefit: Support for scanned documents, broader compatibility.

6. Smarter Hallucination Detection

Improvements: Confidence scores instead of binary yes/no; per-sentence verification; flag unsupported claims instead of rejecting the whole answer.

Expected benefit: Fewer unnecessary regenerations, lower latency, more informative responses.

7. Improve Answer Grading

Improvements: Score usefulness instead of binary classification; accept concise-but-correct answers; factor in retrieval confidence.

Expected benefit: Lower retry rate, faster responses, reduced compute cost.

8. Better Failure Handling

Improvements: Detect insufficient evidence early; stop retrying when retrieval confidence stays low; return a graceful "not found in document" response instead of looping.

Expected benefit: Reduced latency, lower LLM usage, improved user experience.

9. Better Citation System

Improvements: Highlight exact supporting sentences; inline citations; per-citation confidence; clickable source navigation.

Expected benefit: More transparent, verifiable, trustworthy answers.

9.2 Long-Term Improvements

Larger architectural or infrastructure changes, better suited to a dedicated project phase.

1. Multi-Document Retrieval

Improvements: Support multiple PDFs simultaneously; cross-document reasoning; source ranking across documents; document-specific citations.

Expected benefit: Enterprise-scale search, knowledge-base support, document comparison.

2. Streaming Responses

Improvements: Stream tokens in real time; show retrieval progress and intermediate workflow state; incremental citations.

Expected benefit: Better UX, lower perceived latency.

3. Adaptive Retrieval Strategy

Improvements: Detect question type (factoid, multi-hop, summarization, comparison, definition) and adjust retrieval strategy / top-k accordingly.

Expected benefit: Improved accuracy, reduced latency, better scalability.

4. Advanced RAG Evaluation (RAGAS)

Improvements: Faithfulness, Context Precision, Context Recall, Answer Relevancy, Answer Correctness, semantic similarity, automated benchmark datasets.

Expected benefit: Comprehensive measurement, easier strategy comparison, better optimization.

5. Retrieval Evaluation Dashboard

Improvements: Precision@K, Recall@K, MRR, nDCG, hit rate, confidence distributions, failure analysis, query difficulty stats.

Expected benefit: Better monitoring, easier debugging, data-driven optimization.

6. Context Compression

Improvements: Remove duplicate information; compress or summarize lengthy/redundant chunks; keep only what's relevant to the current question.

Expected benefit: Lower token usage, faster generation, improved answer quality.

7. Additional File Types

Improvements: DOCX, PPTX, HTML, Markdown, TXT, CSV, Excel.

Expected benefit: More flexible ingestion, wider applicability.

8. Incremental Indexing

Improvements: Update only changed documents; hash-based duplicate detection; automatic vector-store sync.

Expected benefit: Faster ingestion, better scalability, lower storage overhead.

9. Distributed Vector Database / Production Deployment

Improvements: Migrate from local ChromaDB to a distributed vector database (Pinecone, Weaviate, Milvus, Qdrant); deploy via FastAPI + Docker for scalable, production-grade serving.

Expected benefit: Horizontal scalability, higher availability, enterprise readiness.

10. Improve Retrieval Recall (Structural)

Improvements: Widen the hybrid candidate pool (top 20–30 before reranking); query decomposition for multi-part questions; multi-vector retrieval using titles/headings.

Expected benefit: Higher page recall, better multi-page reasoning, fewer retrieval failures.

Long-Term Vision

Evolve the current self-reflective RAG system into a production-ready document intelligence platform capable of handling multiple document formats, enterprise-scale knowledge bases, adaptive retrieval strategies, confidence-aware answer generation, and comprehensive evaluation — while maintaining low latency and high factual accuracy through continuous self-reflection and retrieval optimization.

Appendix — Current Configuration Reference

A quick-reference snapshot of the production configuration this handbook is based on — useful for a last-minute review immediately before an interview.

Parameter	Value
Embedding Model	BAAI/bge-small-en-v1.5
Embedding Dimension	384
LLM	Llama 3.2 (via Ollama)
Vector Database	ChromaDB
Chunk Size	400 tokens
Chunk Overlap	50 tokens
Retriever	Hybrid (Semantic + BM25)
Reciprocal Rank Fusion (RRF)	Enabled
Cross-Encoder Re-ranking	Enabled (cross-encoder/ms-marco-MiniLM-L-6-v2)
Final Retrieval k	4
Hybrid Candidate k	10
Re-rank Candidate k	20
Final Re-ranked k	5
Hallucination Retry Limit	2
Query Rewrite Retry Limit	2
Orchestration	LangChain + LangGraph
Frontend	Streamlit

This configuration produced the benchmark and evaluation results reported in Chapter 5: 0.44 peak page recall at chunk size 400 during the benchmark sweep, and 0.4075 average page recall on the full production pipeline (which adds cross-encoder re-ranking and the LangGraph self-reflection loop on top of retrieval).